

Material instrucional

ChatOps com Discord e Pipeline Guardian

Do diagnóstico do GitHub Actions ao canal da equipe, com FastAPI, HMAC, adaptadores e governança

Resultado da prática

Ao final, você terá uma integração completa em que o Pipeline Guardian gera um diagnóstico estruturado, o GitHub Actions envia um evento assinado, o FastAPI valida e normaliza, e o Discord recebe um embed com status, evidências, causa provável, impacto e próximos passos.

Sequência da prática

Etapa	Ação	Resultado
1	Preparar repositório, Discord e credenciais	Base segura e canais separados.
2	Executar o prompt principal no Claude Code	Estrutura completa do backend e do workflow.
3	Validar localmente com fixtures	Canal test e canal class funcionando.
4	Expor o FastAPI com Cloudflare Tunnel	GitHub Actions alcança o backend local.
5	Abrir PR positiva e integrar na main	ChatOps estável e auditável.
6	Executar falha intencional em Draft PR	Erro real chega automaticamente ao Discord.

Mensagem central

O agente observa, interpreta e recomenda. O pipeline valida. A política limita. O Discord comunica. A decisão crítica continua sob controle humano.

1. Visão geral e arquitetura da prática

No Dia 1, o Pipeline Guardian já analisou logs, resultados dos jobs e alterações da Pull Request. No Dia 2, a análise não é recriada: o diagnóstico passa a ser transportado para o Discord por uma camada ChatOps.

Pull Request ou push

- > GitHub Actions executa lint, testes e build
- > Pipeline Guardian gera reports/diagnosis.json
- > script Node cria e assina um evento
- > FastAPI valida token, HMAC, timestamp e permissões
- > adaptador normaliza a solicitação
- > formatador cria o embed
- > webhook publica no Discord

O que não muda

O Discord não classifica a falha e não decide o deploy. A análise continua no Pipeline Guardian e a decisão continua na política determinística. O canal apenas entrega a informação ao time.

Componente	Responsabilidade
Pipeline Guardian	Investigar logs, diff, evidências, causa provável e recomendação.
Política	Determinar blocked, eligible_for_staging ou requires_human_approval.
send-chatops-event.mjs	Transportar o diagnosis.json sem reinterpretar a falha.
FastAPI	Validar, normalizar e orquestrar o envio.
DiscordEmbedFormatter	Adaptar a resposta estruturada ao formato do canal.
DiscordWebhookClient	Publicar o embed e tratar timeout e rate limit.

2. Preparar o repositório e a branch

Comece sempre pela main estável. O objetivo desta prática é alterar o canal de comunicação, não corrigir uma base já quebrada.

```
git switch main
git pull --ff-only origin main
git status

npm ci
npm run lint
npm run test
npm run build

# Branch da implementação real
git switch -c feat/chatops-discord
```

Checkpoint

Todos os comandos da main devem passar antes de criar a integração. A branch feat/chatops-discord será uma PR normal e, depois da validação, deverá ser integrada à main.

3. Configurar o Discord antes do código

3.1 Criar os canais

1. No servidor do Discord, crie um canal de texto privado chamado pipeline-guardian-test.
2. Restrinja a visualização para você ou para a equipe responsável pela prática.
3. Crie um segundo canal chamado ci-alertas-turma, destinado à demonstração com os alunos.
4. Restrinja a criação e a edição de webhooks aos administradores do servidor.

3.2 Criar um webhook em cada canal

1. Abra Editar canal > Integrações > Webhooks.
2. Crie um webhook chamado Pipeline Guardian.
3. Selecione o canal correto e copie a URL.
4. Repita o processo no canal da turma.
5. Guarde as duas URLs em local seguro; elas funcionam como credenciais.

Segredo crítico

Não cole a URL do webhook em chats, prints, commits, README ou GitHub Actions. Se ela for exposta, exclua o webhook e crie outro.

4. Gerar os segredos e criar o arquivo local

CHATOPS_SHARED_SECRET e CHATOPS_API_TOKEN não são fornecidos pelo Discord. Você mesmo gera esses valores e usa os mesmos dados no backend e no GitHub Actions.

```
# Execute duas vezes e guarde cada saída
openssl rand -hex 32
openssl rand -hex 32
```

Crie o arquivo chatops/.env depois que a estrutura for gerada. Se a pasta ainda não existe, apenas guarde os valores e continue para o prompt principal.

```
# chatops/.env
CHATOPS_SHARED_SECRET=<PRIMEIRA_SAIDA_DO_OPENSSL>
CHATOPS_API_TOKEN=<SEGUNDA_SAIDA_DO_OPENSSL>
DISCORD_WEBHOOK_URL_TEST=https://discord.com/api/webhooks/ID/TOKEN
DISCORD_WEBHOOK_URL_CLASS=https://discord.com/api/webhooks/ID/TOKEN
ALLOWED_REPOSITORY=IA-para-DEVs-SCTEC-T2/pipeline-guardian-agent-demo
MAX_TIMESTAMP_SKEW_SECONDS=300
```

Separação correta das configurações

O backend conhece as URLs do Discord e a allowlist. O GitHub Actions conhece apenas a URL pública do FastAPI, o target lógico e os dois segredos compartilhados. CHATOPS_ENDPOINT_URL não precisa estar em chatops/.env.

5. Prompt principal para o Claude Code

Execute o prompt nesta etapa, antes da teoria detalhada. Use Claude Opus com effort high. Comece em Plan Mode, revise o plano e depois permita as alterações manualmente.

```
/clear
/model opus
/effort high
# Ative Plan Mode com Shift + Tab
```

Orientação de execução

O modelo deve implementar e testar, mas não deve fazer commit, push, merge ou alterar Secrets e Variables por conta própria.

Você é um especialista em Python, FastAPI, arquitetura com adaptadores, Discord Webhooks, GitHub Actions, Node.js, segurança com HMAC, Cloudflare Tunnel, testes automatizados e integração de agentes de IA.

Trabalhe no repositório atual pipeline-guardian-agent-demo.

CONTEXTO

0 Dia 1 já implementou:

- aplicação CopaFigurinhas;
- lint, testes e build;
- workflow .github/workflows/ci.yml;
- workspace automation;
- Pipeline Guardian investigativo;
- reports/diagnosis.json e reports/diagnosis.md;
- política determinística de decisão.

Não recrie o Pipeline Guardian e não copie sua lógica para Python.

0 objetivo do Dia 2 é transportar o diagnóstico já produzido para o Discord.

ANTES DE ALTERAR

1. confirme que a branch atual é feat/chatops-discord;
2. inspecione .github/workflows/ci.yml;
3. inspecione automation/src/ci-diagnose.mjs;
4. inspecione o schema real de reports/diagnosis.json;
5. identifique como o comentário da PR é habilitado ou desabilitado;
6. apresente um plano curto;
7. implemente somente depois do plano;
8. não faça commit;
9. não faça push;
10. não crie Pull Request;
11. não altere regras da aplicação CopaFigurinhas;
12. não implemente Slack, Teams ou e-mail;
13. não enfraqueça token, HMAC, timestamp ou allowlist;
14. não imprima secrets nem URLs completas de webhook.

OBJETIVO

Criar esta integração:

GitHub Actions

- > Pipeline Guardian gera diagnosis.json
- > script Node envia evento assinado
- > FastAPI valida e normaliza
- > formatador cria embed
- > webhook publica no Discord

ARQUITETURA

Crie:

```
chatops/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── config.py
│   ├── models.py
│   └── adapters/
│       ├── __init__.py
│       ├── inbound/github_actions.py
│       └── outbound/discord.py
│   ├── clients/discord_webhook.py
│   ├── security/signature.py
│   └── services/chatops_service.py
├── fixtures/
│   ├── test_failure.json
│   ├── dependency_failure.json
│   ├── environment_failure.json
│   └── permission_failure.json
├── scripts/send_fixture.py
├── tests/
├── requirements.txt
├── .env.example
└── README.md
```

Crie também:

- automation/src/send-chatops-event.mjs;

- automation/tests/send-chatops-event.test.mjs usando Vitest;
- ajustes mínimos em .github/workflows/ci.yml;
- documentação em docs/CHATOPS-DISCORD.md.

CONFIGURAÇÃO LOCAL

O Settings deve localizar chatops/.env de forma robusta a partir do caminho do próprio módulo, sem depender do diretório corrente.

Use pydantic-settings e SecretStr.

Variáveis obrigatórias:

- CHATOPS_SHARED_SECRET;
- CHATOPS_API_TOKEN;
- DISCORD_WEBHOOK_URL_TEST;
- DISCORD_WEBHOOK_URL_CLASS;
- ALLOWED_REPOSITORY;
- MAX_TIMESTAMP_SKEW_SECONDS=300.

O valor real da allowlist deve ser:

IA-para-DEVs-SCTEC-T2/pipeline-guardian-agent-demo

ENTRADA DO FASTAPI

Endpoint:

POST /webhooks/github-actions

Headers:

- Authorization: Bearer <token>;
- X-ChatOps-Timestamp;
- X-ChatOps-Signature: sha256=<hmac>.

Payload:

- event_type: pipeline_diagnosis;
- source: github_actions;
- repository;
- branch;
- commit_sha;
- run_id;
- run_url;
- target: test ou class;
- diagnosis: objeto real do Pipeline Guardian.

Não invente um segundo schema incompatível com diagnosis.json.

CONTRATO NORMALIZADO

Produza internamente:

- channel: discord;
- channel_id;
- user_id: github-actions;
- command: pipeline-diagnosis;
- arguments com repository, branch, commit_sha, run_url e diagnosis;
- response_target: test ou class;
- request_id.

SEGURANÇA

Implemente:

1. bearer token;
2. HMAC SHA-256 sobre "timestamp.corpo_bruto";
3. hmac.compare_digest;
4. rejeição de timestamps com mais de 300 segundos;
5. allowlist exata de repository;
6. allowlist de target: test ou class;
7. nenhuma URL do Discord aceita no payload;
8. URLs do Discord somente no backend;
9. nenhum segredo em logs ou respostas;
10. allowed_mentions com parse vazio;
11. limites de tamanho do Discord;
12. request_id em todos os logs.

DISCORD

Use incoming webhooks, não bot.

Crie DiscordEmbedFormatter separado do cliente HTTP.

O embed deve conter:

- título e resumo;
- status e tipo de falha;
- sinal;
- causa provável;
- impacto;
- próximos passos;
- risco e confiança;
- recomendação do agente;
- decisão da política;

- repositório, branch e commit;
- link para a execução;
- request_id no footer;
- timestamp.

Cores:

- blocked: 0xD64545;
- requires_human_approval: 0xE9B949;
- eligible_for_staging: 0x2FBF71;
- fallback: 0x5865F2.

Respeite:

- título 256;
- descrição 4096;
- no máximo 25 fields;
- field name 256;
- field value 1024;
- soma máxima 6000 caracteres;
- um embed por mensagem.

CLIENTE DISCORD

Use httpx.AsyncClient.

Use ?wait=true.

Configure timeout.

Trate 429 usando retry_after.

Faça no máximo 3 tentativas.

Não registre a URL do webhook.

SCRIPT NODE

automation/src/send-chatops-event.mjs deve:

- ler reports/diagnosis.json;
- criar o payload de transporte;
- usar GITHUB_REPOSITORY, GITHUB_SHA, GITHUB_RUN_ID, GITHUB_SERVER_URL, GITHUB_HEAD_REF ou GITHUB_REF_NAME;
- criar timestamp Unix em segundos;
- assinar exatamente o JSON enviado;
- usar CHATOPS_ENDPOINT_URL como URL-base;
- acrescentar /webhooks/github-actions uma única vez;
- usar CHATOPS_SHARED_SECRET, CHATOPS_API_TOKEN e CHATOPS_TARGET;
- aceitar somente test ou class;
- falhar com mensagem segura quando o backend responder erro;
- não imprimir secrets.

WORKFLOW

Depois de reports/diagnosis.json existir, execute send-chatops-event.mjs.

Passe:

- CHATOPS_ENDPOINT_URL por vars;
- CHATOPS_TARGET por vars;
- CHATOPS_SHARED_SECRET por secrets;
- CHATOPS_API_TOKEN por secrets.

Adicione um step seguro chamado Validate ChatOps configuration que informe apenas configurado/ausente para endpoint, target, shared secret, API token e diagnosis.json. Não imprima valores.

Evite uma condição silenciosa como vars.CHATOPS_ENDPOINT_URL != '' que deixe o step Notify Discord como skipped sem explicação.

O step de notificação deve executar em Pull Requests quando diagnosis.json existir; o próprio script deve produzir mensagem segura se uma configuração estiver ausente. Mantenha continue-on-error durante o laboratório.

A notificação não pode alterar o CI Gate.

Preserve Job Summary e artefatos.

Defina AUTOMATION_ALLOW_PR_COMMENT=false na demonstração do Dia 2.

FIXTURES E CLI

Crie quatro cenários coerentes com o schema real:

1. test_failure: AssertionError na regra duplicateCopies;
2. dependency_failure: Cannot find module 'dotenv';
3. environment_failure: variável obrigatória ausente;
4. permission_failure: 403 ou permission denied.

chatops/scripts/send_fixture.py deve aceitar:

- --scenario com escolhas sem extensão .json;
- --target test ou class;
- --repository opcional;
- --endpoint como URL-base, por exemplo http://127.0.0.1:8000.

O script deve acrescentar /webhooks/github-actions uma única vez.

Quando --repository for informado, sobrescreva o repository do envelope e do

diagnosis com o mesmo valor.
 Quando não for informado, use nesta ordem:
 1. ALLOWED_REPOSITORY carregado de chatops/.env;
 2. repository existente na fixture como fallback.

O script deve imprimir apenas status, request_id e message_id.

TESTES
 Use pytest e mocks de httpx no backend.
 Use Vitest para o transportador Node.
 Os testes devem cobrir:

- normalização;
- token inválido;
- assinatura inválida;
- timestamp expirado;
- repository não autorizado;
- target não autorizado;
- seleção test/class;
- cores e truncamento do embed;
- allowed_mentions vazio;
- 429 com retry;
- falha de rede;
- nenhum segredo nos logs;
- /health;
- endpoint válido retornando 202;
- URL-base sem duplicação do caminho;
- CLI --repository;
- repositório real da integração.

Evite testes herméticos que validem apenas senai/copa-figurinhas em fixtures e conftest. Inclua ao menos um teste com o repositório real:
 IA-para-DEVs-SCTEC-T2/pipeline-guardian-agent-demo

COMANDOS DE VALIDAÇÃO
 Execute:

- npm run lint;
- npm run test;
- npm run build;
- npm run test:automation;
- chatops/.venv/bin/python -m pytest -q chatops/tests.

Não use node --test para os testes Vitest.
 Não dependa do pytest global.

Mostre ao final:

- árvore dos arquivos;
- comandos executados;
- testes realizados;
- exemplo do contrato normalizado;
- exemplo do embed;
- alterações do workflow;
- riscos e passos manuais no Discord e GitHub.

Não faça commit.
 Não faça push.

6. Entendendo o que o prompt criou

6.1 Um backend, vários canais: arquitetura com adaptadores

Mesmo usando apenas Discord, a separação por adaptadores impede que o núcleo seja contaminado pelo formato do canal. Um futuro adaptador para Teams, e-mail ou dashboard reutilizaria o mesmo contrato.

Camada	Papel no projeto
Entrada	POST /webhooks/github-actions recebe corpo e headers.
Adaptador de entrada	Converte o evento do GitHub Actions para contrato normalizado.
Serviço ChatOps	Orquestra permissões, formatação e envio.
Formatador Discord	Transforma diagnóstico estruturado em embed.

Camada	Papel no projeto
Cliente Discord	Executa a chamada HTTP e trata erros de rede.
Pipeline Guardian	Permanece responsável pela interpretação técnica.

Princípio central

O canal é uma interface. Ferramentas retornam dados ao agente; o agente produz a resposta estruturada; o formatador adapta essa resposta ao canal.

6.2 Contrato normalizado

```
{
  "channel": "discord",
  "channel_id": "pipeline-guardian-test",
  "user_id": "github-actions",
  "command": "pipeline-diagnosis",
  "arguments": {
    "repository": "IA-para-DEVs-SCTEC-T2/pipeline-guardian-agent-demo",
    "branch": "demo/slide-15-16-test-diagnosis",
    "commit_sha": "1c72dea",
    "run_url": "https://github.com/.../actions/runs/123456789",
    "diagnosis": { "...": "diagnostico real" }
  },
  "response_target": "test",
  "request_id": "req-2026-07-14-001"
}
```

O payload nunca leva a URL do webhook. Ele transporta apenas o target lógico test ou class. O backend escolhe a URL autorizada.

6.3 Segurança do fluxo

Controle	O que protege
Bearer token	Autentica o emissor.
HMAC SHA-256	Comprova integridade e posse do segredo.
Timestamp	Evita replay de requisições antigas.
ALLOWED_REPOSITORY	Impede eventos de repositórios não autorizados.
Target allowlist	Limita a saída a test ou class.
request_id	Relaciona GitHub, FastAPI e Discord.
allowed_mentions	Evita @everyone ou menções acidentais.

6.4 Formatação do embed

Campo do diagnóstico	Elemento no Discord
summary	Descrição principal.
pipelineStatus / failureType	Status e categoria.
signal	Sinal técnico observado.
probableCause	Hipótese de causa.
impact	Efeito sobre a aplicação ou entrega.

Campo do diagnóstico	Elemento no Discord
nextSteps	Ações recomendadas.
riskLevel / confidence	Risco e força da hipótese.
policyDecision	Cor e decisão exibida.
run_url	Link para a execução.
request_id	Rodapé de rastreabilidade.

7. Criar o ambiente Python e revisar os arquivos

```
python3 -m venv chatops/.venv
source chatops/.venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r chatops/requirements.txt

find chatops -maxdepth 4 -type f | sort
git status --short
git diff --stat
```

Crie o arquivo real a partir do exemplo e preencha com os valores gerados anteriormente:

```
cp chatops/.env.example chatops/.env
code chatops/.env
```

Proteção do arquivo

chatops/.env deve estar ignorado. Confirme com: `git check-ignore -v chatops/.env`. O arquivo nunca deve aparecer no commit.

8. Executar o FastAPI corretamente

Use um terminal exclusivo para o backend. O parâmetro `--env-file` evita que o Pydantic procure apenas um `.env` na raiz e não encontre `chatops/.env`.

```
source chatops/.venv/bin/activate

python -m uvicorn chatops.app.main:app --host 0.0.0.0 --port 8000 --reload --env-file chatops/.env
```

Parâmetro	Significado
chatops.app.main:app	Importa o objeto FastAPI chamado app.
--host 0.0.0.0	Aceita conexões locais e as encaminhadas pelo túnel.
--port 8000	Porta do backend.
--reload	Reinicia ao detectar alterações; use apenas em desenvolvimento.
--env-file chatops/.env	Carrega as variáveis locais corretas.

Em outro terminal, valide:

```
curl http://127.0.0.1:8000/health
# Esperado: {"status":"ok"}
```

```
# Documentacao interativa
# http://127.0.0.1:8000/docs
```

Respostas esperadas no navegador

GET / retorna 404 porque a aplicação não possui rota raiz. GET /webhooks/github-actions retorna 405 porque o endpoint aceita POST. Esses códigos não indicam falha do servidor.

9. Testar primeiro no computador local

9.1 Teste direto do webhook

```
set -a
source chatops/.env
set +a

curl -sS -X POST "${DISCORD_WEBHOOK_URL_TEST}?wait=true" -H "Content-Type:
application/json" -d '{
  "content": "Teste inicial do Pipeline Guardian",
  "allowed_mentions": {"parse": []}
}' | python -m json.tool
```

Se essa mensagem não chegar, o problema está no webhook do Discord, antes do FastAPI.

9.2 Fixture positiva no target test

Mantenha o FastAPI aberto no Terminal 1. No Terminal 2, execute a fixture usando a URL-base. Não acrescente /webhooks/github-actions e não use .json no nome do cenário.

```
python chatops/scripts/send_fixture.py --scenario dependency_failure --target test --
repository IA-para-DEVs-SCTEC-T2/pipeline-guardian-agent-demo --endpoint
http://127.0.0.1:8000
```

Resultado esperado

```
status:      202 accepted
request_id:  <uuid>
message_id:  <id-do-discord>
```

Por que o túnel não é necessário aqui?

O emissor e o FastAPI estão no mesmo computador. 127.0.0.1 aponta para a sua própria máquina. O túnel será necessário apenas quando o emissor for o runner remoto do GitHub.

9.3 Fixture no target class

```
python chatops/scripts/send_fixture.py --scenario test_failure --target class --
repository IA-para-DEVs-SCTEC-T2/pipeline-guardian-agent-demo --endpoint
http://127.0.0.1:8000
```

O backend deve selecionar DISCORD_WEBHOOK_URL_CLASS. Revise a mensagem antes de usar o canal da turma durante a aula.

9.4 Outros cenários locais

```
python chatops/scripts/send_fixture.py --scenario test_failure --target test --repository IA-para-
DEVs-SCTEC-T2/pipeline-guardian-agent-demo --endpoint http://127.0.0.1:8000
python chatops/scripts/send_fixture.py --scenario environment_failure --target test --repository IA-para-
DEVs-SCTEC-T2/pipeline-guardian-agent-demo --endpoint http://127.0.0.1:8000
```

```
python chatops/scripts/send_fixture.py --scenario permission_failure --target test --repository IA-para-DEVs-SCTEC-T2/pipeline-guardian-agent-demo --endpoint http://127.0.0.1:8000
```

9.5 Testes automatizados corretos

```
npm run lint
npm run test
npm run build
npm run test:automation

chatops/.venv/bin/python -m pytest -q chatops/tests
```

Dois erros evitados

Os testes do transportador usam Vitest; não execute `node --test`. Os testes Python devem usar o `venv` do ChatOps; não dependa de um `pytest` global que pode não ter FastAPI instalado.

10. Expor o FastAPI para o GitHub Actions

Um runner hospedado pelo GitHub é outra máquina. Para ele, `127.0.0.1` significa o próprio runner, não o seu computador. Por isso, o FastAPI precisa de uma URL pública HTTPS.

```
# Terminal 1 - FastAPI
python -m uvicorn chatops.app.main:app --host 0.0.0.0 --port 8000 --reload --env-file chatops/.env

# Terminal 2 - Cloudflare Quick Tunnel
cloudflared tunnel --url http://localhost:8000
```

O `cloudflared` exibirá uma URL semelhante a `https://algum-endereco.trycloudflare.com`. Mantenha FastAPI e túnel abertos durante todo o workflow.

```
# Terminal 3 - validar acesso externo
curl https://SUA-URL-ATUAL.trycloudflare.com/health
# Esperado: {"status":"ok"}
```

Quick Tunnel muda de endereço

Ao reiniciar `cloudflared`, uma nova URL pode ser gerada. Atualize somente a Repository Variable `CHATOPS_ENDPOINT_URL`. Não altere `chatops/.env`, os webhooks ou os segredos.

11. Configurar GitHub Secrets e Repository Variables

A separação entre Secrets e Variables é obrigatória. Se `CHATOPS_ENDPOINT_URL` for criada como Secret, `vars.CHATOPS_ENDPOINT_URL` ficará vazia e o step poderá ser ignorado.

Tipo	Nome	Valor
Secret	CHATOPS_SHARED_SECRET	Mesmo valor do <code>chatops/.env</code> .
Secret	CHATOPS_API_TOKEN	Mesmo valor do <code>chatops/.env</code> .
Variable	CHATOPS_ENDPOINT_URL	URL-base atual do túnel, sem o caminho final.
Variable	CHATOPS_TARGET	test inicialmente; class para a turma.

Na interface: Settings > Secrets and variables > Actions. Cadastre os dois primeiros na aba Secrets e os dois últimos na aba Variables.

```
# Opcional: configuracao com GitHub CLI
gh secret set CHATOPS_SHARED_SECRET
gh secret set CHATOPS_API_TOKEN

gh variable set CHATOPS_ENDPOINT_URL --body "https://SUA-URL-ATUAL.trycloudflare.com"
gh variable set CHATOPS_TARGET --body "test"

# Verificacao segura
gh variable list
gh secret list
```

Organização correta

Variables esperados:
CHATOPS_ENDPOINT_URL
CHATOPS_TARGET
OPENAI_MODEL

Secrets esperados:
CHATOPS_API_TOKEN
CHATOPS_SHARED_SECRET

12. Versionar a implementação e abrir a PR positiva

Antes do commit, revise todos os arquivos e confirme que nenhuma credencial foi versionada.

```
git status --short
git diff --stat
git diff

grep -RinE "discord.com/api/webhooks|CHATOPS_SHARED_SECRET|CHATOPS_API_TOKEN" chatops
automation .github docs --exclude='.env' --exclude='.env.example' || true

git check-ignore -v chatops/.env
```

Você pode organizar os commits por responsabilidade:

```
git add chatops
git commit -m "feat: add Discord ChatOps backend and adapters"

git add automation/src/send-chatops-event.mjs automation/tests/send-chatops-
event.test.mjs .github/workflows/ci.yml
git commit -m "ci: send Pipeline Guardian diagnoses to ChatOps"

git add docs/CHATOPS-DISCORD.md .gitignore
git commit -m "docs: add Discord ChatOps setup guide"

git push -u origin feat/chatops-discord
```

12.1 Pull Request

Campo	Valor
base	main
compare	feat/chatops-discord
tipo	Create pull request - não Draft
título	feat: deliver Pipeline Guardian diagnoses to Discord

Objetivo

Adiciona uma integração ChatOps para enviar diagnósticos estruturados do Pipeline Guardian a canais do Discord.

Arquitetura

GitHub Actions -> Pipeline Guardian -> diagnosis.json -> evento assinado
-> FastAPI -> contrato normalizado -> DiscordEmbedFormatter -> Webhook

Principais alterações

- backend FastAPI e adaptadores;
- bearer token, HMAC, timestamp e allowlist;
- formatador de embeds e cliente Discord;
- targets test e class;
- fixtures e testes automatizados;
- transportador Node;
- integração no workflow sem alterar o CI Gate.

Validações

- [] npm run lint
- [] npm run test
- [] npm run build
- [] npm run test:automation
- [] chatops/.venv/bin/python -m pytest -q chatops/tests
- [] fixture target test
- [] fixture target class
- [] nenhuma credencial versionada
- [] Notify Discord retorna HTTP 202

13. Validar a comunicação real na PR positiva

Com FastAPI e túnel ativos, um novo push na PR deve executar o fluxo real. Se não houver arquivos a alterar, crie um commit vazio para gerar uma execução atual.

```
git switch feat/chatops-discord
git pull --ff-only origin feat/chatops-discord

git commit --allow-empty -m "test: validate Discord ChatOps notification"
git push origin feat/chatops-discord
```

No GitHub, abra Checks > Diagnose (Pipeline Guardian) e confirme:

```
Validate ChatOps configuration      success
Notify Discord (ChatOps)          success

[chatops] evento aceito - HTTP 202
[chatops] request_id: ...
[chatops] message_id: ...
```

No terminal do FastAPI, deve aparecer:

```
POST /webhooks/github-actions HTTP/1.1 202 Accepted
```

Quando fazer o merge

Integre feat/chatops-discord na main somente depois que os testes estiverem verdes, o step não estiver skipped, o backend retornar 202 e a mensagem chegar ao canal test.

Depois, use Squash and merge e atualize sua main local:

```
git switch main
```

```
git pull --ff-only origin main
```

14. Exemplo positivo: pipeline saudável no Discord

A PR positiva valida que a infraestrutura funciona sem depender de uma falha. O embed pode indicar pipeline saudável ou elegível para staging, conforme o diagnosis.json real.

Campo	Exemplo positivo
Status	success
Decisão	eligible_for_staging
Risco	low
Confiança	high
Próximo passo	Prosseguir para staging conforme a política.
CI Gate	success

Interpretação

O Discord informa que os gates passaram. Ele não executa deploy e não altera a política.

15. Exemplo negativo: erro intencional comunicado ao Discord

Depois que a integração está na main, reutilize a branch de demonstração que contém a falha intencional na regra duplicateCopies. Essa branch deve permanecer Draft e nunca ser integrada.

15.1 Atualizar a branch demo com a main

```
git switch demo/slide-15-16-test-diagnosis
git pull --ff-only origin demo/slide-15-16-test-diagnosis

git fetch origin
git merge origin/main

git status
git push origin demo/slide-15-16-test-diagnosis
```

Direção do merge

Como você está na branch demo, git merge origin/main traz a integração estável da main para a demo. A main não recebe a falha. O comando perigoso seria estar na main e mesclar a demo.

15.2 Draft Pull Request

Campo	Valor
base	main
compare	demo/slide-15-16-test-diagnosis
tipo	Create draft pull request
título	demo: Pipeline Guardian analyzes business rule failure

Campo	Valor
regra	DO NOT MERGE

Demonstração educacional

Esta Draft Pull Request contém uma falha intencional na regra de cálculo de figurinhas repetidas.

Resultado esperado

- Quality: aprovado
- Tests: reprovado
- Build: aprovado
- Diagnose: aprovado
- Notify Discord: aprovado
- CI Gate: reprovado

O diagnóstico deve apresentar `AssertionError`, causa provável em `duplicateCopies`, impacto no relatório e decisão `blocked`.

****DO NOT MERGE****

15.3 Resultado esperado no pipeline

Job	Status	Motivo
Quality	Sucesso	A falha não viola ESLint.
Tests	Falha	<code>duplicateCopies</code> retorna valor incorreto.
Build	Sucesso	O código continua compilável.
Diagnose	Sucesso	O agente processa a falha e gera <code>diagnosis.json</code> .
Notify Discord	Sucesso	O backend aceita o evento e publica o embed.
CI Gate	Falha	Testes reprovados bloqueiam a entrega.

15.4 Resultado esperado no Discord

```
Pipeline Guardian - pipeline falhou
Status: failed / blocked
Tipo de falha: test
Sinal: test:AssertionError
Causa provável: duplicateCopies passou a contar a figurinha original como repetida
Impacto: o relatório exibe uma repetida a mais
Próximos passos:
1. reproduzir com npm run test -w backend
2. revisar backend/src/services/report.js
3. restaurar quantity - 1
Risco: medium
Confiança: high
Decisão da política: blocked
Branch: demo/slide-15-16-test-diagnosis
```

Conceito central

Uma notificação bem-sucedida não aprova o pipeline. O Discord comunica a análise; o CI Gate continua bloqueando a entrega.

15.5 Repetir a demonstração no dia da aula

Mantenha FastAPI e túnel ativos, atualize CHATOPS_ENDPOINT_URL se o Quick Tunnel mudou e configure CHATOPS_TARGET=class quando estiver pronto para a turma.

```
gh variable set CHATOPS_TARGET --body "class"

git switch demo/slide-15-16-test-diagnosis
git pull --ff-only origin demo/slide-15-16-test-diagnosis

git commit --allow-empty -m "demo: send simulated pipeline failure to class Discord channel"

git push origin demo/slide-15-16-test-diagnosis
```

16. Diagnóstico de problemas - soluções já incorporadas

Sintoma	Causa provável	Correção
ValidationError ao iniciar FastAPI	chatops/.env não foi carregado.	Use python -m uvicorn ... --env-file chatops/.env.
GET / retorna 404	Não existe rota raiz.	Use /health ou /docs.
GET no webhook retorna 405	A rota aceita somente POST.	Use send_fixture.py ou GitHub Actions.
ConnectError na fixture	FastAPI foi encerrado ou porta incorreta.	Mantenha o Terminal 1 aberto e teste /health.
404 na fixture	Endpoint completo foi passado e o script duplicou o caminho.	Passe somente http://127.0.0.1:8000.
invalid choice test_failure.json	O CLI aceita o nome sem extensão.	Use --scenario test_failure.
unrecognized --repository	CLI antigo sem a flag.	Implemente/atualize o script conforme o prompt.
pytest não encontra FastAPI	pytest de outro ambiente.	Use chatops/.venv/bin/python -m pytest.
node --test falha com Vitest	Runner incorreto.	Use npm run test:automation.
Notify Discord aparece skipped	CHATOPS_ENDPOINT_URL ausente em Repository Variables ou condição silenciosa.	Cadastre em Variables e use Validate ChatOps configuration.
403 repository not allowed	ALLOWED_REPOSITORY ou fixture usa repo fictício.	Use o owner/repository real e a flag --repository.
GitHub não alcança o backend	Túnel fechado ou URL antiga.	Reinicie o túnel, teste /health e atualize a Variable.
Fixture local falha com túnel ativo	Problema local independente do túnel.	Teste 127.0.0.1 e limpe variáveis antigas do shell.
HTTP 401	API token divergente.	Use o mesmo valor no .env e no Secret.
HTTP 403 assinatura inválida	Shared secret ou corpo assinado divergente.	Compare o secret e assine exatamente o JSON enviado.
HTTP 502	FastAPI aceitou, mas Discord falhou.	Revise webhook, target e cliente Discord.
429 Discord	Muitas mensagens.	Respeite retry_after e reduza simulações.

16.1 Ordem correta de investigação

1. Teste diretamente o webhook do Discord.
2. Inicie o FastAPI com --env-file e teste /health.
3. Execute uma fixture local para target test.
4. Execute a fixture para target class.
5. Execute as suítes automatizadas.
6. Inicie o túnel e teste a URL pública em /health.

7. Confirme gh variable list e gh secret list.
8. Gere uma nova execução de PR.
9. Procure o mesmo request_id no GitHub, FastAPI e Discord.

Não altere várias camadas ao mesmo tempo

A progressão webhook -> FastAPI local -> túnel -> GitHub Actions reduz o número de causas possíveis em cada etapa.

17. Checklist antes da aula

Item	Verificação
main	Atualizada e saudável.
PR de integração	Mesclada na main.
Draft PR demo	Aberta, marcada DO NOT MERGE.
FastAPI	Rodando na porta 8000 com --env-file.
Tunnel	Ativo e /health público retorna 200.
Variables	CHATOPS_ENDPOINT_URL atual e CHATOPS_TARGET=class.
Secrets	CHATOPS_SHARED_SECRET e CHATOPS_API_TOKEN corretos.
Allowlist	ALLOWED_REPOSITORY usa owner/repository real.
Discord	Webhook test e class válidos.
Testes	npm run test:automation e pytest do venv verdes.
Logs	Nenhum segredo aparece.
Rerun	Commit vazio pronto para disparar a demo.

18. Desafio para casa - ChatOps Pipeline Assistant

No desafio, você evoluirá a integração para consultar o estado do pipeline por API ou mock. A ação crítica continua controlada por política e não deve ser executada automaticamente.

Requisito	Resultado
Comando pipeline-status	Consulta workflow, branch e conclusão.
GitHubApiClient e MockGitHubClient	Rede real quando disponível e testes reproduzíveis.
Contrato normalizado	Canal e origem não contaminam o serviço.
Reuso do Pipeline Guardian	Não criar classificador duplicado.
Política	Falha bloqueia; sucesso permite staging; produção exige aprovação.
Discord	Embed com status, sinal, causa, próxima ação e link.
Testes	Sem chamadas reais ao Discord.

Você é um especialista em Python, FastAPI, Discord, GitHub API, arquitetura com adaptadores, agentes de IA, segurança de webhooks, testes automatizados e didática de engenharia de software.

Trabalhe no repositório pipeline-guardian-agent-demo já utilizado nos Dias 1 e 2.

ANTES DE ALTERAR

1. confirme uma branch exclusiva, por exemplo feat/chatops-pipeline-assistant;
2. inspecione o workspace automation e o Pipeline Guardian;
3. inspecione o backend chatops criado no Dia 2;
4. inspecione .github/workflows/ci.yml;
5. identifique o schema real de diagnosis.json;
6. apresente um plano breve;
7. não faça commit;
8. não faça push;
9. não crie Pull Request;
10. não execute deploy ou rollback real.

OBJETIVO

Implementar um ChatOps Pipeline Assistant usando Discord.

O sistema deve aceitar uma solicitação normalizada ou evento do GitHub Actions, consultar o estado do pipeline, reutilizar o Pipeline Guardian, produzir uma resposta normalizada e publicar um embed no Discord.

FLUXO

Evento ou comando

- > FastAPI
- > validação de origem e autorização
- > adaptador de entrada
- > contrato normalizado
- > consulta GitHub API ou mock
- > Pipeline Guardian / diagnosis.json existente
- > política determinística
- > formatador Discord
- > webhook do Discord

ARQUITETURA

Mantenha as camadas do Dia 2 e adicione somente o necessário:

- chatops/app/adapters/inbound/chatops_command.py;
- chatops/app/clients/github_api.py;
- chatops/app/services/pipeline_status_service.py;
- fixtures e testes para GitHub API mock.

COMANDOS

Implemente:

1. pipeline-status
argumentos: repository e branch;
2. pipeline-diagnosis
argumentos: repository e run_id ou diagnosis.

GITHUB

Crie uma porta de consulta com:

- GitHubApiClient, usado quando GITHUB_TOKEN estiver presente;
- MockGitHubClient, usado em testes e demonstrações.

Recupere, quando disponível:

- workflow;
- run_id;
- status;
- conclusion;
- branch;
- commit_sha;
- jobs;
- URL da execução.

REGRAS

- falha de gate -> blocked;
- todos os gates verdes -> eligible_for_staging;
- produção -> requires_human_approval;
- nenhuma ação crítica automática;
- nenhuma duplicação do agente;
- logs com request_id e sem secrets.

SEGURANÇA

Reutilize bearer token, HMAC, timestamp, allowlist, target e allowed_mentions.
Não aceite URL de Discord no payload.

TESTES

Use mocks para GitHub e Discord.

Teste sucesso, falha de testes, falha de build, permissão e API indisponível.
Execute npm run lint, npm run test, npm run build,

```
npm run test:automation e chatops/.venv/bin/python -m pytest -q chatops/tests.
```

ENTREGA

Mostre a árvore, contratos, exemplos de request/response, testes e riscos.
Não faça commit nem push.

19. Conclusão

A prática conecta o raciocínio do agente a um canal operacional sem transformar o Discord em fonte de verdade ou executor de ações críticas. O Pipeline Guardian interpreta; o GitHub Actions executa os gates; a política decide; o FastAPI protege e normaliza; o Discord entrega a resposta ao time.

Aprendizado	Aplicação
Adaptadores	Permite trocar o canal sem duplicar o núcleo.
Contrato normalizado	Reduz acoplamento entre GitHub, backend e Discord.
HMAC, token e allowlist	Protegem um endpoint público.
Tunnel	Conecta temporariamente o runner remoto ao backend local.
Variables e Secrets	Separam configuração operacional de credenciais.
Fixtures	Validam cada camada antes da integração ponta a ponta.
Draft PR com falha	Demonstra erro real sem contaminar a main.
Governança	Mantém merge, deploy e produção sob controle.

Resultado final

Você possui um fluxo em que uma mudança defeituosa gera falha de teste, o agente relaciona log e diff, a política bloqueia a entrega e o diagnóstico chega automaticamente ao Discord com evidências e próximos passos.